

# DSC 40B Discussion 5 Worksheet Solutions

Friday, May 1, 2026

Imagine you are a tutor, and you need to explain something to someone who has only a vague understanding of what's going on. Do your best to get the point across. **ALTERNATE**.

If you're not sure, ask your peer if they know how to explain it, then REPEAT the explanation back. Remember, the point of the exercise is to **vocalize** your thoughts.

**If you are both unsure, make sure to come to/tutor for help! Do not use ChatGPT or any other LLM! The point of discussion is to prepare you for the exams.**

Have one person in the pair open the Discussion 1 Gradescope assignment. After both you **and** your partner finish a problem **and explain it**, answer the corresponding multiple choice/short answer questions for both versions.. You will receive instant feedback on the questions, whether you are correct or not. If your answer is incorrect, please go back to the question to understand it and feel free to ask tutors about anything you are confused about. Once you have finished the entire worksheet or discussion is ending, turn in your assignment and add your partner to the submission.

## Quick Icebreaker

Introduce yourself to your partner by sharing name, year, major, etc. What is your favorite off-campus food spot in San Diego? (Do not spend more than 2 minutes!)

## Questions

### Question 1:

#### First Person:

```
def mystery1(arr):
    count = 0
    for x in arr:
        y = random.random()
        if x % 2 == 0:
            count += 1
            print(y)
    return count
```

For the code snippet above, is it more appropriate to analyze its time complexity with average or expected time complexity. Explain your reasoning.

*Expected time is not really necessary here because the random value  $y$  is printed but does not affect how many times the loop runs. The loop is deterministic and always checks every element in  $arr$ , so the runtime is always linear,  $O(n)$ , regardless of the random calls.*

## Second Person:

```
def mystery2(arr):  
    count = 0  
    for x in arr:  
        y = random.random()  
        count += 1  
        if random.random() * 2 < 1:  
            return count  
    return count
```

For the code snippet above, is it more appropriate to analyze its time complexity with average or expected time complexity. Explain your reasoning.

*Expected time is more appropriate here because the random condition can cause the function to return early. The number of loop iterations depends on random chance, so we would analyze the expected number of iterations before the function stops.*

## Question 2:

### Second Person:

Permutation sort is a sorting algorithm that deterministically cycles through all  $n!$  ways to arrange the elements of an input list of size  $n$ , and terminates once it has found a permutation in which the list is sorted. Bogosort is a sorting algorithm that randomly shuffles the elements of an input list and terminates once the list is sorted. Match each of these algorithms with the type of runtime analysis that makes the most sense to perform.

- ☐ Expected for both
- ☐ Expected for permutation sort and average case for bogosort
- ☒ ~~Average case for permutation sort and expected for bogosort~~
- ☐ Average case for both

*Permutation sort is deterministic and its runtime depends on the data itself, so it makes sense to analyze its average case complexity. Bogosort is a random algorithm so it makes sense to analyze its expected runtime complexity.*

### First Person:

For each of the following algorithms, determine whether it makes sense to compute the average case or the expected time complexity:22222222222222

- Performing a binary search for the number 53 on a sorted list
- Inserting  $n$  elements into an empty BST (not self balancing) and then searching for 53

- ☐ Expected for both
- ☐ Expected for binary search and average case for BST
- ☐ Average case for binary search and expected for BST

☒ Average case for both

*Neither algorithm is random, but each has a runtime that depends on what the input data looks like, so it makes sense to look at the average case complexity for both. For example, binary search (for 53) will terminate faster on a list that only contains 53's as opposed to a list that only contains 0's. Binary tree insertion can be very slow if the tree becomes unbalanced, which depends solely on the incoming data.*

### Question 3:

#### First Person:

Given `arr` is an arbitrary list with random integers, you perform randomized quickselect (where the pivot index is selected randomly) on `arr`. To best analyze the asymptotic time complexity, should you compute average or expected time complexity? Explain your reasoning.

- ☐ Average  
☒ Expected

*Expected time complexity is more appropriate because the algorithm itself uses randomness when choosing the pivot. Even if the input array is fixed, the runtime can vary depending on the random pivot choices.*

#### Second Person:

Given `arr` is an arbitrary list with random integers, you perform fixed quickselect (where the pivot index is always the first element) on `arr`. To best analyze the asymptotic time complexity, should you compute average or expected time complexity? Explain your reasoning.

- ☒ Average  
☐ Expected

*Average time complexity is more appropriate because the algorithm itself is deterministic and always chooses the first element as the pivot. The only randomness comes from the assumed distribution of inputs, so we average over possible input orders.*

### Question 4:

#### Second Person:

True or False? Quicksort always runs in  $O(n \log n)$  time because each partition step takes linear time. Explain your reasoning.

- ☐ True

☒ False

*Each partition step is linear, but if the pivot repeatedly creates very unbalanced partitions, the total runtime can become  $O(n^2)$ .*

### First Person:

True or False: In randomized quicksort, the input array does not need to be randomly ordered for the algorithm to have good expected runtime.? Explain your reasoning.

☒ True

☐ False

*The randomness comes from the pivot choices, not from the input order, so even a fixed input can have expected  $O(n \log n)$  runtime.*

### Question 5:

Example Quicksort Tracing:

[2, 4, 1, 3]

→ [2, 1, 3, 4]

→ [1, 2, 3, 4]

### First Person:

For the list below, run fixed quicksort with the pivot as the last element and in-place partitioning (Approach #2 in lecture) on it and trace the list in the same way as the example above (after each “layer” of partitions).

[4, 1, 6, 2, 7, 3, 5]

[4, 1, 6, 2, 7, 3, 5]

→ [4, 1, 2, 3, 5, 6, 7]

→ [1, 2, 3, 4, 5, 6, 7]

→ [1, 2, 3, 4, 5, 6, 7]

→ [1, 2, 3, 4, 5, 6, 7]

### Second Person:

For the list below, run fixed quicksort with the pivot as the last element and in-place partitioning (Approach #2 in lecture) on it and trace the list in the same way as the example above (after each “layer” of partitions).

[6, 2, 7, 1, 5, 3, 4]

[6, 2, 7, 1, 5, 3, 4]

→ [2, 1, 3, 4, 5, 7, 6]

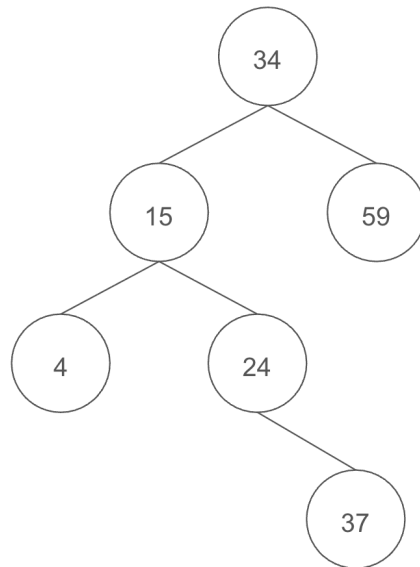
→ [2, 1, 3, 4, 5, 6, 7]

→ [1, 2, 3, 4, 5, 6, 7]

→ [1, 2, 3, 4, 5, 6, 7]

Question 6:

**Second Person:**



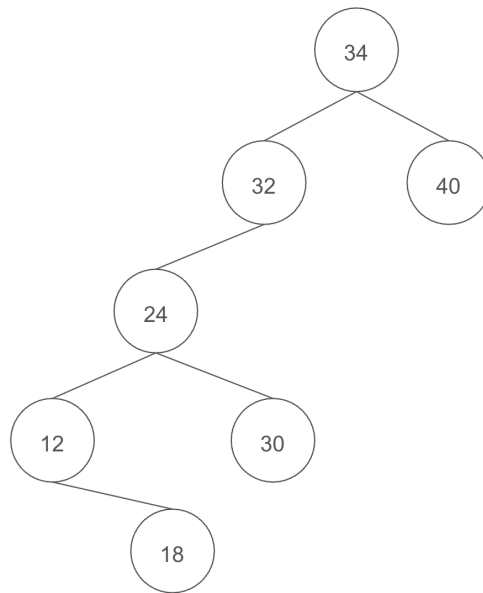
Is the tree picture above a valid binary search tree? Explain your reasoning.

☐ Valid

☒ ~~Not Valid~~

*The 37 node is supposed to go in the right subtree of the root, as 37 is greater than 34.*

**First Person:**

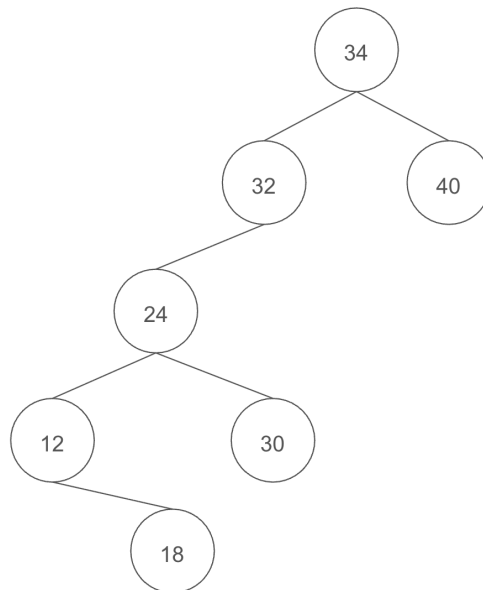


Is the tree picture above a valid binary search tree? Explain your reasoning.

- ☒ Valid  
☐ Not Valid

*Every node's left subtree only comprises of values smaller than itself and every node's right subtree only comprises of values greater than itself.*

#### Question 7:



**First Person:**

Given the binary search tree above, insert the following sequence of numbers and draw the final tree.

[28, 39, 36, 10]

**Second Person:**

Given the binary search tree above, insert the following sequence of numbers and draw the final tree.

[33, 20, 19, 42]

### Question 8:

**Second Person:**

Is the hash function pictured below a good hash function? Explain your reasoning.

```
def hash_A(x, m): # m is table size
    return int(m * x + 8) % 10
```

*No, hash\_a is not a good hash function, because it will only hash into the first 10 buckets. This results in a distribution of hashes that is not relatively uniform and will eventually cause insertions of linear time into those first 10 buckets.*

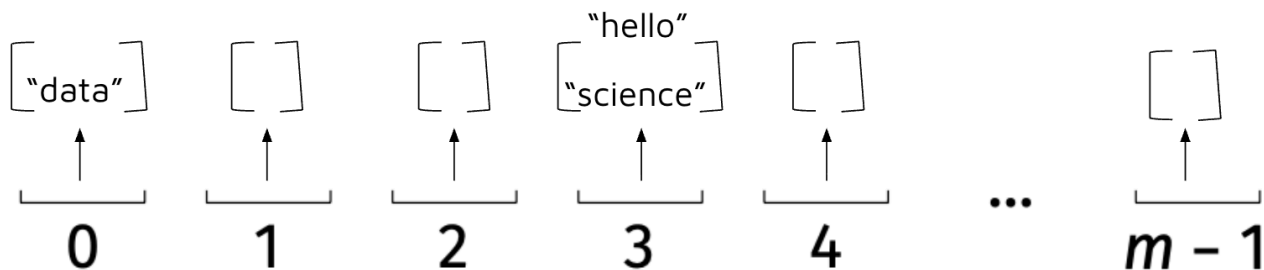
**First Person:**

Is the hash function pictured below a good hash function? Explain your reasoning.

```
def hash_B(x, m): # m is table size
    return int(random.random() * x - 7) % m
```

*No, hash\_b is not a good hash function, because it is not deterministic. The random() call will result in the same element being possibly hashed into different buckets when called separate times.*

### Question 9:



Hash table after hashing and inserting the sequence ["data", "science", "hello"].

You are given an empty hash table initialized with  $m=5$  buckets that handles collisions with separate chaining (as pictured above) and the below hash function.

```
def hash_func(x, m): # m is table size (5)
    return (3 * x + 8) % m
```

### First Person:

Draw the hash table in the same way as above after inserting the following sequence of numbers.

[4, 6, 1, 9, 3, 8, 0, 2, 10]

```
0: [4, 9]
1: [6, 1]
2: [3, 8]
3: [0, 10]
4: [2]
```

### Second Person:

Draw the hash table in the same way as above after inserting the following sequence of numbers.

[1, 6, 11, 16, 21, 2, 7, 12]

```
0: []
1: [1, 6, 11, 16, 21]
2: []
3: []
4: [2, 7, 12]
```